

Critique exercise — a senior review of an AI-written dependency graph

CSC322 Group E. Subject: the module in `transcript.md`, produced by ChatGPT on 23 July in response to a request for a "complete, production-quality" dependency-tracking module.

Reviewed as we would review a pull request from a competent colleague who was working fast: say what is right, then what is wrong, then what it costs, then what we did instead.

1. What is correct

Credit where it is due, because most of this module is right.

The data structure is the right one. Two adjacency maps, forward and reverse, is exactly what the problem needs. Insertion is amortised $O(1)$; membership is $O(1)$; the reverse map is what makes "who is affected" answerable without a scan. We built the same shape.

The traversal is the right algorithm. Depth-first search with a three-colour marking (`visited`, `visiting`, everything else), pushing each vertex after its descendants, is the standard topological sort, and detecting a back edge to a grey vertex is the standard cycle detection. It is textbook and it is correct.

GetAffected is right. Breadth-first closure over the reverse edges gives exactly the transitive dependents, and the `if (affected.Add(d))` idiom deduplicates and guards against non-termination in one line. We would have merged this into the ordering pass, but as a standalone it is clean.

Range expansion answers the empty-cell question correctly. When we pushed on this (transcript, question 2), the answer was right and for the right reason: the graph is keyed by *address*, so a cell that is empty today still has an identity. That is a real insight and it is the argument that makes range expansion defensible at all.

A first-year implementation would get the reverse map or the three-colour marking wrong. This does not.

2. Where it is subtly wrong

These are the ones that pass a demo and fail in a results office.

2.1 The representation invariant is broken on every formula edit

`SetDependencies` clears `_precedents[cell]` and rebuilds it — and never touches `_dependents`. The invariant these two maps must satisfy is

$$p \in \text{_precedents}[c] \iff c \in \text{_dependents}[p]$$

and after a single edit it does not hold. If `C1 = A1` is changed to `C1 = B1`, `A1` is removed from `C1`'s precedents but `C1` remains in `A1`'s dependents, forever.

The consequences run from bad to worse:

- `GetAffected("A1")` reports `C1`, so the engine recomputes a cell that cannot have changed. Wasted work — survivable.
- The stale edge is never collected. A cell edited a thousand times accumulates a thousand stale reverse edges.
- **A cycle that has been broken is still reported.** Set `A1 = B1` and `B1 = A1` (a cycle), then fix `B1 = 5`. `_precedents[B1]` no longer contains `A1`, so the cycle is gone — but nothing else changed, and any code that walks `_dependents` still sees the ring. The user fixes the error and the error stays.

This is the defect we would block a pull request on. It is not a performance opinion; it is a data structure that lies about its own contents.

What we do: `DependencyGraph.ClearPrecedents` walks the old precedent list and removes the back-edge from each before clearing. The invariant is stated in `adt-specifications.md` §7 as the first clause, precisely because it is the one that is easy to break. `EditingAFormulaRemovesTheEdgesItNoLongerNeeds` and `BreakingTheCycleRestoresEveryCell` are the tests.

2.2 The cycle path is not deterministic

`Visit` iterates `_precedents[cell]`, a `HashSet<string>`, whose enumeration order is unspecified and depends on insertion history and hash codes. Two workbooks with the same cycle, built in different orders, report different paths. So does the same workbook after a rebuild.

The brief asks for the cycle to be reported "with the exact cycle (e.g. A1 to B3 to C7 back to A1)". A path that changes between runs is not a bug you can file.

Worse, the `path` list is shared across sibling branches and never truncated on the way back up: by the time a cycle is found, `path` contains every cell the search has walked through, not the cycle. The reported "cycle" is a prefix of noise followed by the real ring.

What we do: the ring is extracted from the frame stack (so it is exactly the ring), reversed into "refers to" order, and rotated to begin at the smallest address. `TheCycleIsReportedTheSameWayWhicheverCellClosedIt` builds the same cycle from two different typing orders and requires identical output.

2.3 Cycles are reported by throwing

`CircularReferenceException` abandons the recalculation. Three problems:

- A workbook with two independent cycles reports one and stops. Ours reports both — `TwoIndependentCyclesAreBothReported`.
- The cells *not* in the cycle are left stale, because the exception unwound before they were evaluated. The user gets an exception and a workbook whose other numbers are silently wrong.
- The brief explicitly asks for the cycle to be *reported to the client*, "never with a crash". A propagating exception from an API call is, from a client's point of view, a crash it has to catch.

What we do: a back edge is recorded and **not followed**. The traversal finishes, everything outside the ring is ordered and evaluated correctly, the ring's members get `#CIRC!`, and the cycles arrive in `CalculationResult.Cycles`.

2.4 Recursion

`Visit` recurses one frame per cell of depth. A chain of dependent cells is common in imported workbooks — a running total down a column is exactly that — and 20,000 is not unusual. At that depth this overflows.

`StackOverflowException` in .NET **cannot be caught**. The process dies. For a class library embedded in a results portal, that is the worst possible failure mode: not an error the client handles, but a server that disappears.

What we do: an explicit frame stack. `ADeepChainDoesNotOverflowTheStack` and `ALongCycleDoesNotOverflowTheStack` hold the line at 20,000 cells.

3. Where it falls short of the specification

3.1 It recalculates the whole workbook on every edit

`GetRecalculationOrder` iterates every key of `_precedents` and topologically sorts the entire graph. The brief's target is *"a single cell edit in a workbook of 100,000 cells ... must propagate within 50 milliseconds"*. This is $O(V + E)$ over the whole workbook for every keystroke.

The follow-up exchange in the transcript is the instructive part. Asked how to fix it, it proposed intersecting the full order with the affected set — which reduces *the work reported*, not the work done. Only when asked directly whether that reduced the work did it give the correct answer: run the search from the changed cell over the dependent edges and reverse the post-order. **That correct answer is one sentence of prose at the end of a four-question conversation; the "complete implementation" it was asked for does not contain it.**

Our measured figure for the same shape of workbook is 0.32 ms worst case (`benchmarks.md` §3). We did not benchmark the AI's version, which we should have; our estimate from the full-recalculation figure is ~75 ms, i.e. it would fail the target — but on a faster machine it might pass, which is the genuinely dangerous outcome.

3.2 Range expansion does not survive contact with the brief

Correct (§1), and it does not scale. `=SUM(A1:A100000)` in a hundred formulas is ten million `HashSet<string>` entries — roughly a gigabyte with string keys.

Asked about it, the module's author suggested capping the expandable range size or "storing large ranges separately and checking them linearly", and concluded "for most spreadsheets the expansion approach is fine". Both suggestions are reasonable; neither was implemented; and "for most spreadsheets" is not a specification. A cap in particular does not degrade gracefully — it makes correctness depend on range size, so the engine works until someone selects a whole column.

What we do: a 64×64 spatial index, with an always-scanned fallback list only for ranges spanning more than 1,024 blocks. `=SUM(B2:B45)` is one index entry. The empty-cell property that makes expansion attractive is preserved, because the index is keyed by geometry rather than by cell identity.

3.3 String keys

`"Sheet1!B2"` as a vertex identifier means a string hash and a comparison on every adjacency lookup, and an allocation every time one is constructed. The graph is walked once per dependent per keystroke.

It also has no defined behaviour for case (`Sheet1` vs `sheet1`) or for sheet names containing `!`, both of which the engine must handle.

What we do: dense `int` identifiers, interned per (sheet, column, row). Adjacency is array indexing; sheet-name comparison happens once, at intern time. The identifiers also make the traversal marks flat arrays, which enables §3.4.

3.4 The traversal allocates two hash sets per edit

`GetRecalculationOrder` allocates `visited`, `visiting` and `order` on every call. In a 100,000-cell workbook the first two grow to 100,000 entries — per keystroke — and then become garbage.

What we do: the mark arrays are fields, reused, and cleared by incrementing a generation counter rather than by writing to them. This is the single largest contributor to the 0.32 ms figure, and it is invisible unless you are thinking about the target.

3.5 Silence about what it does not do

The module is presented as "complete". It has no notion of which sheet a cell is on beyond a string prefix, no removal of a vertex, no accounting for a cell that is referenced but does not exist, and no statement of its representation invariant — the thing that would have made §2.1 obvious to its author.

That last point is the general lesson. The module has no specification, so nothing anchors the code to an intent, and the invariant break is invisible.

4. What we did differently, in one table

Aspect	The AI module	CalcEngine	Consequence
Vertex identity	<code>"Sheet1!B2"</code> string	interned <code>int</code>	array adjacency; flat mark arrays
Edit cost	topological sort of the whole workbook	DFS from the edited cell only	0.32 ms vs an estimated ~75 ms
Traversal marks	two <code>HashSet</code> s per call	generation-stamped arrays	no per-edit allocation
Ranges	expanded to one edge per cell	64×64 spatial index	<code>=SUM(B2:B45)</code> is one entry, not 44
Cycle reporting	throws, first cycle only	recorded in the result, all cycles	non-cycle cells still evaluated

Aspect	The AI module	CalcEngine	Consequence
Cycle path	non-deterministic, includes non-cycle cells	canonical, exactly the ring	reproducible message
Recursion	yes	explicit stack	survives a 20,000-cell chain
Edge removal	reverse edges leak	both directions cleared	a fixed cycle stays fixed
Specification	none	AF and RI, in code and in docs	the invariant break would have been visible

5. What we take from the exercise

It wrote better code than the average first draft, and it would have failed the project. Not on style, and not on the algorithm — on two constraints that were in the brief and not in the prompt: the 100,000-cell target and "never with a crash or an infinite loop".

The correct answer was available, and only on demand. Four questions in, it described precisely the design we had built. It had the knowledge; what it did not have was any reason to apply it, because nothing in the request told it the edit cost mattered. It optimised for the request as stated — short, clear, correct on a small example — which is exactly what it should do, and exactly why the engineer, not the tool, owns the result.

The invariant is the review tool. Every defect in §2 is a violation of a property we could write down in one line. We found §2.1 by asking "what is the invariant between these two maps, and where is it restored?" — a question the module cannot answer because it never states one. That is the habit we took from this exercise into the rest of the project, and it is why every ADT in `CalcEngine.Core` carries an abstraction function and a representation invariant in its XML documentation.